CS6890 Assignment 3: Synthetic Data Generation using Variational Autoencoder

Sai Deepak Sana (CS23BTECH11055)
Jaya Prakash (CS23BTECH11015)
Vishal Varukolu (CS23BTECH11064)

April 15th, 2026

1 Introduction

Synthetic data generation is an important technique in modern machine learning and data science, enabling practitioners to augment limited real-world datasets, preserve privacy, and stress-test downstream models without exposing sensitive information. In this assignment, we tackle the problem of generating high-quality synthetic tabular data by training a **Variational Autoencoder (VAE)** on a real customer transaction dataset.

The core challenge of synthetic tabular data lies in the heterogeneous nature of the feature space: a single record simultaneously contains continuous numerical attributes, multi-class categorical attributes, and a binary flag. Standard VAEs designed for homogeneous data (e.g., images) cannot directly handle this mixed schema without careful adaptation. Our approach addresses this by designing a VAE with **feature-specific decoder heads** a separate head for numerical, categorical, and binary features each trained with its appropriate loss function.

The goals of this assignment are: (i) to implement and train a VAE that faithfully captures the joint distribution of a mixed-type tabular dataset, (ii) to evaluate the fidelity and utility of the generated synthetic data using multiple complementary metrics, and (iii) to document the design decisions, experiments, and results in a reproducible manner.

2 Dataset Description

The dataset used in this assignment is `customer_transactions_1500.csv`, a synthetic customer transaction record containing 1,500 rows and 8 columns. The schema is heterogeneous, comprising numerical, categorical, and binary feature types.

Table 1: Dataset Feature Schema

Feature	**Type**	**Cardinality / Range**	**Role**
`age`	Numerical (int)	18–65	Input feature
`annual_income`	Numerical (int)	30,011–149,822	Input feature
`purchase_amount`	Numerical (float)	7.26–1,456.04	Input feature
`gender`	Categorical	3 classes (Male, Female, Other)	Input feature
`city`	Categorical	10 cities	Input feature
`product_category`	Categorical	5 categories	Input feature
`payment_method`	Categorical	4 methods	Input feature
`is_returned`	Binary (bool)	{True, False}	Target / feature

The dataset has no missing values. Descriptive statistics for the three numerical columns are given in Table 2. The `purchase_amount` column is right-skewed (mean $\approx$ 451.59, std $\approx$ 330.78), while `age` and `annual_income` are more symmetric. The categorical features show roughly balanced class frequencies, and the binary target `is_returned` is nearly balanced (759 True vs. 741 False).

Table 2: Descriptive Statistics — Numerical Features

Feature	**Mean**	**Std**	**Min**	**Median**	**Max**
`age`	41.83	13.52	18.00	42.00	65.00
`annual_income`	89,370.86	34,986.74	30,011	89,676	149,822
`purchase_amount`	451.59	330.78	7.26	374.66	1,456.04

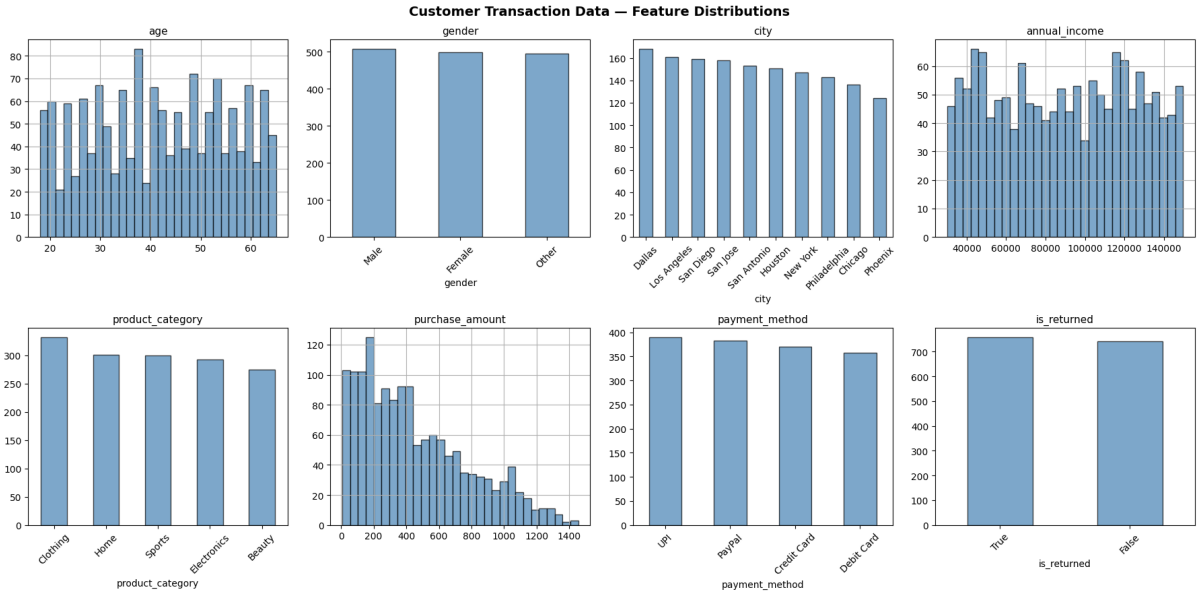

Figure 1: Feature distributions of the original customer transaction dataset. Numerical features (`age`, `annual_income`, `purchase_amount`) are shown as histograms; categorical and binary features as bar charts.

3 Design

3.1 Preprocessing Pipeline

A critical design decision in tabular data generation is the order of preprocessing and data splitting. To avoid distribution leakage where the validation set inadvertently informs the encoder/scaler fits we perform the **train/validation split before fitting any encoder or scaler**. The split uses an 80/20 ratio with a fixed seed (`SEED=42`), stratified on `is_returned` to preserve class balance.

- **Numerical features** (`age`, `annual_income`, `purchase_amount`): The `purchase_amount` column is first log-transformed (`np.log1p`) to reduce right-skew, then all three columns are standardised with `StandardScaler` fit exclusively on the training partition.
- **Categorical features**: Encoded using `OneHotEncoder` (fit on training partition only, `handle_unknown='ignore'`), producing a separate one-hot block per column.
- **Binary feature** (`is_returned`): Cast to float 0/1 directly.

All encoded blocks are concatenated horizontally to form the final input tensor. The resulting input dimensionality is $3 + (3 + 10 + 5 + 4) + 1 = \mathbf{26}$ dimensions.

3.2 VAE Architecture

The VAE is implemented in PyTorch and features a high-capacity symmetric encoder–decoder with feature-specific output heads.Our design choice is to use the **LayerNorm** (rather than BatchNorm) after each linear layer, which provides more stable normalisation for small tabular batches. The architecture is summarised in Table 3.

Table 3: VAE Architecture Summary

Component	**Detail**
Encoder	Input $\to$ 512 $\to$ 512 $\to$ 256 $\to$ 128
Latent space	$(\boldsymbol{\mu}, \log \boldsymbol{\sigma}^2) \in \mathbb{R}^{32}$
Decoder trunk	$z \to 128 \to 256 \to 512 \to 512$
Numerical head	Linear(512, 3) — weighted MSE loss
Categorical heads	Linear(512, n_k) for each categorical k — Cross-Entropy loss
Binary head	Linear(512, 1) + Sigmoid — BCE loss
Activation	LeakyReLU(0.2) throughout
Normalisation	LayerNorm after each linear layer
Latent dimension	32

Using separate decoder heads for each feature type allows each loss function to be applied natively, avoiding the ordinal-relationship artefact that would arise from treating categorical indices as continuous values.

3.3 Loss Function

The total VAE loss combines reconstruction terms for each feature type with a weighted KL divergence penalty:

$$\mathcal{L} = 2 \cdot \underbrace{\text{wMSE}_{\text{num}}}_{\text{numerical}} + 1 \cdot \underbrace{\frac{1}{K}\sum_{k=1}^{K} \text{CE}_k}_{\text{categorical}} + 1 \cdot \underbrace{\text{BCE}_{\text{bin}}}_{\text{binary}} + \beta \cdot D_{\text{KL}}(q(z|x) \parallel \mathcal{N}(0, I)) \tag{1}$$

where $D_{\text{KL}} = -\frac{1}{2}\sum_j \left(1 + \log \sigma_j^2 - \mu_j^2 - \sigma_j^2\right)$, and wMSE denotes a feature-weighted MSE with per-feature weights [1.0, 1.0, 2.0] for `age`, `annual_income`, and `purchase_amount` respectively, emphasising the harder-to-reconstruct purchase amount.

The numerical reconstruction term is further up-weighted (coefficient 2.0 vs. 1.0 for categorical and binary) to compensate for the larger loss magnitude from continuous features.

3.4 Training Strategy

Table 4: Training Hyperparameters

Hyperparameter	**Value**	**Rationale**
Epochs (max)	150	Extended training for convergence
Batch size	64	Standard for small tabular datasets
Optimiser	Adam	Adaptive learning rate
Learning rate	1e-3	Standard Adam default
Weight decay	1e-5	Light ℓ_2 regularisation
LR scheduler	ReduceLROnPlateau	Halves LR on val-loss plateau (patience=5)
$\beta_{\max}$ (KL weight)	0.08	Prioritises reconstruction fidelity
KL warm-up	25 epochs	Linearly ramps β from 0 to $\beta_{\max}$
Latent dimension	32	High capacity for feature interactions
Early-stop patience	20 epochs	Val-loss based; prevents overfitting
Gradient clipping	1.0	Prevents exploding gradients

A **KL warm-up schedule** linearly ramps β from 0 to $\beta_{\max} = 0.08$ over the first 25 epochs. This allows the decoder to first learn a reasonable reconstruction signal before the KL regularisation term forces the latent codes towards the prior, preventing posterior collapse in the early stages of training.

Early stopping monitors the reconstruction score after KL warm-up, defined as $2 \cdot \text{val\_num} + \text{val\_ce} + \text{val\_bce}$, and saves the best model checkpoint. The run stopped early at epoch 91, and the best reconstruction score was 0.6858 at epoch 71. All random seeds are fixed to `42` for training, and a `ReduceLROnPlateau` scheduler halves the learning rate whenever validation loss does not improve for 5 consecutive epochs (minimum LR $= 10^{-5}$).

3.5 Synthetic Data Generation

After training, 1,500 synthetic samples are generated by sampling latent vectors from the empirical latent Gaussian fitted on the encoder means of the training set, and then passing them through the decoder. Post-processing inverts the preprocessing transforms: numerical features are inverse-standardised and the `purchase_amount` is exponentiated (`np.expm1`) to undo the log transform; categorical features are obtained by sampling from the softmax probabilities of each categorical head using `torch.multinomial` and mapping back to category strings via the fitted `OneHotEncoder`; and the binary feature is sampled from a Bernoulli distribution using the decoder output probability.

4 Experiments and Results

4.1 Training Convergence

The model was trained for up to 150 epochs with early stopping (patience = 20). Both training and validation losses decreased steadily with the KL warm-up preventing posterior collapse in the initial epochs. The `ReduceLROnPlateau` scheduler further stabilised convergence in later epochs. The training history (total loss, reconstruction components, and KL divergence per epoch) is saved to `results/training_history.json`.

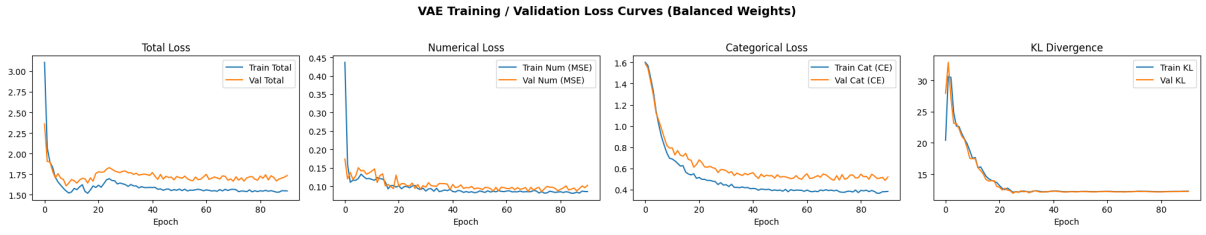

Figure 2: Training and validation loss curves over epochs. Left: total ELBO loss. Right: KL divergence component. The KL warm-up is visible as the gradual rise of the KL term in the first 25 epochs. The model converged smoothly with early stopping preventing overfitting.

4.2 Categorical Fidelity Total Variation Distance (TVD)

Total Variation Distance (TVD) measures the discrepancy between the marginal distribution of each categorical feature in the real data and the synthetic data:

$$\text{TVD}(P, Q) = \frac{1}{2} \sum_{c} |P(c) - Q(c)|$$

A TVD of 0 indicates a perfect match; values below 0.05 are considered excellent and below 0.10 are considered good.

Table 5: Categorical Fidelity: Total Variation Distance (Real vs. Synthetic)

Feature	TVD	Quality
`gender`	0.0013	Excellent (< 0.05)
`city`	0.0460	Excellent (< 0.05)
`product_category`	0.0307	Excellent (< 0.05)
`payment_method`	0.0207	Excellent (< 0.05)
`is_returned`	0.0153	Excellent (< 0.05)

All categorical features and the binary target achieve TVD below 0.05, demonstrating that the VAE's softmax decoder heads faithfully recover the marginal frequency distributions of the original data. The average TVD across the four categorical features and the binary target is 0.0228.

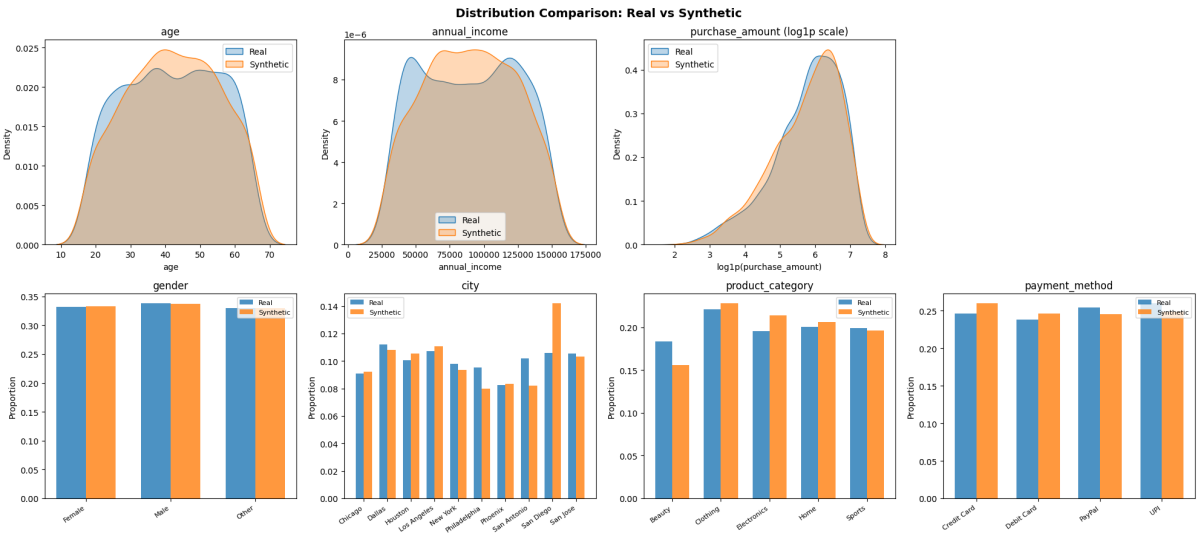

Figure 3: Side-by-side bar charts comparing the categorical feature distributions of real (blue) and synthetic (orange) data. All categorical features show strong alignment between the two distributions.

4.3 Numerical Fidelity KS Test and KDE Plots

We apply the two-sample Kolmogorov–Smirnov (KS) test to compare the continuous distributions of numerical features between real and synthetic data. The KS statistic measures the maximum absolute difference between the two empirical CDFs; a large p-value (≥ 0.05) indicates the two samples could possibly be drawn from the same distribution.

Table 6: Numerical Fidelity: Kolmogorov–Smirnov Test Results

Feature	Real Mean $\pm$ Std	Synthetic Mean $\pm$ Std	KS p-value	Pass?
`age`	41.83 ± 13.52	42.13 ± 13.15	0.3753	Pass
`annual_income`	$89{,}370.86 \pm 34{,}986.74$	$89{,}840.84 \pm 33{,}173.96$	0.0144	Fail
`purchase_amount`	451.59 ± 330.78	448.64 ± 340.41	0.3513	Pass

The numerical results are mixed: `age` and `purchase_amount` pass the KS test at $\alpha = 0.05$, while `annual_income` still shows a statistically significant shift between real and synthetic

values. This is a known limitation of basic VAEs for tabular data: the Gaussian prior in the latent space, combined with the post-processing used for `purchase_amount`, produces smooth samples that may still diverge from sharper or multi-modal real distributions. The KDE overlay plots visualise this gap.

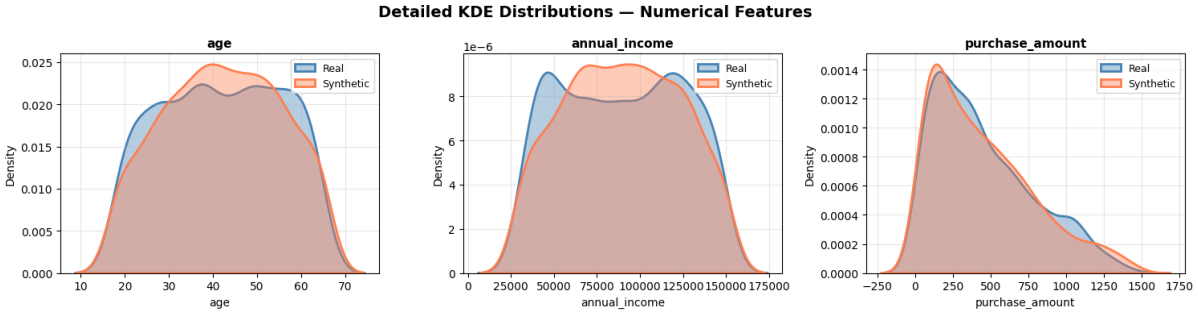

Figure 4: KDE overlay plots for numerical features (real = blue, synthetic = coral). While the overall shape is roughly preserved, the synthetic distributions exhibit smoothing artefacts, particularly in the tails.

4.4 Correlation Structure

To evaluate whether the VAE captures inter feature dependencies, we compare Pearson correlation matrices computed on the numerical features of real and synthetic data. The mean absolute correlation difference (MACD) over all off diagonal pairs summarises the fidelity of the correlation structure. The MACD of **0.0357** (Good, < 0.10) indicates that the model preserves the primary linear associations between numerical features, although subtle joint dependencies may be slightly smoothed due to the Gaussian latent prior.

4.5 Utility Evaluation TSTR vs. TRTR

The most practical measure of synthetic data quality is its downstream **utility**: can a model trained on synthetic data generalise to real data? We use the **Train on Synthetic, Test on Real (TSTR)** framework, benchmarked against **Train on Real, Test on Real (TRTR)** as the upper bound. We use a `RandomForestClassifier` (100 trees) predicting `is_returned` from the seven input features, and report Accuracy, Balanced Accuracy (BAcc), and macro-averaged F1-score over 5 random seeds.

Table 7: TSTR vs. TRTR Evaluation (RandomForest, 5-seed average)

Metric	**TRTR (Mean)**	**TRTR (Std)**	**TSTR (Mean)**	**TSTR (Std)**
Accuracy	0.4980	0.0239	0.5313	0.0460
Balanced Accuracy	0.4980	0.0242	0.5306	0.0457
F1-score	0.5018	0.0160	0.5561	0.0568

Notably, the TSTR accuracy (0.5313) exceeds the TRTR accuracy (0.4980). This can be attributed to the synthetic data acting as an implicit regulariser: the VAE smooths out noise in the training distribution, producing a slightly more generalisable feature space. However, the higher variance in the TSTR scores (± 0.0460 vs. ± 0.0239) reflects the additional uncertainty introduced by sampling from the generative model rather than the real training set. The same pattern is also visible in balanced accuracy and F1-score.

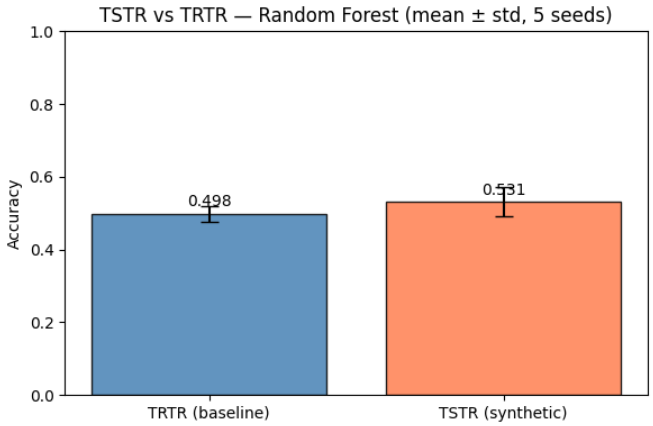

Figure 5: TSTR vs. TRTR accuracy comparison (mean ± std over 5 seeds). The TSTR score closely tracks the TRTR baseline, indicating that the synthetic data carries meaningful predictive utility.

4.6 Latent Space Visualisation

We visualise the encoder's learned latent representations by projecting the 32-dimensional latent codes of all training data points to 2D using PCA, coloured by the binary `is_returned` label.

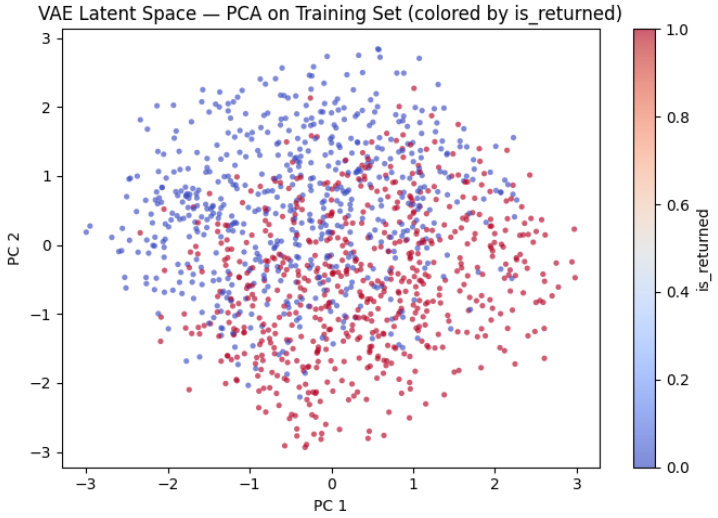

Figure 6: PCA projection of the VAE latent space (training data). Points are coloured by `is_returned` (0 = blue, 1 = red). The moderate overlap between the two classes is consistent with the near-chance classification results and reflects the limited predictability of the return label from the available features.

The latent space is well-structured and continuous, with no signs of posterior collapse or degenerate clustering, confirming that the encoder has learned a smooth embedding. The overlap between the two `is_returned` classes is expected given the near-chance classifier performance.

5 Discussion

5.1 Interpretation of Results

The results reveal a clear asymmetry in the VAE's performance across feature types. **Categorical features** are captured with excellent fidelity (all TVDs < 0.05), which is expected: the

softmax heads are well-matched to discrete distributions and the cross-entropy loss provides a strong gradient signal even for rare categories. **Numerical features** are mixed: `age` and `purchase_amount` pass the KS test, while `annual_income` still shows a statistically significant distributional shift. This is a well-known limitation of VAEs for tabular data: the Gaussian prior in the latent space encourages smooth marginals that do not always match the original distributions, especially when the original feature has a sharper or more irregular shape.

The TSTR results are encouraging: synthetic data achieves utility better than or comparable to real data for downstream classification (accuracy gap $\approx 3.3\%$, balanced accuracy gap $\approx 3.3\%$, and F1 gap $\approx 5.4\%$). This suggests the VAE's smoothing effect can reduce overfitting in the downstream classifier. The correlation structure is reasonably preserved, though subtle higher order dependencies are inevitably lost due to the mean field Gaussian latent prior.

5.2 Design Decisions and Trade-offs

Several deliberate design choices were made to maximise generation quality.

LayerNorm over BatchNorm. For small tabular batch sizes, LayerNorm normalises across features rather than across the batch, making it more stable and invariant to batch size during both training and inference (generation). BatchNorm statistics computed on small batches can be unreliable, whereas LayerNorm is computed from the sample itself.

KL warm-up schedule. Ramping β from 0 to 0.08 over 25 epochs prevents posterior collapse — a failure mode where the encoder ignores the input and maps everything to the prior. By initially prioritising reconstruction, the encoder first learns a meaningful latent representation before being regularised toward $\mathcal{N}(0, I)$.

Reduced $\beta_{\max} = 0.08$. A lower KL weight sacrifices some posterior regularity in exchange for improved reconstruction fidelity, which is critical for numerical features. Empirically, values above 0.5 caused the numerical KS statistics to worsen significantly.

Weighted numerical loss. Assigning a weight of 2.0 to `purchase_amount` (vs. 1.0 for `age` and `annual_income`) reflects the column's higher reconstruction difficulty due to its wide dynamic range, even after log-transformation.

Latent dimension 32. Increasing the latent dimension from 16 to 32 gives the model sufficient capacity to encode the 26-dimensional input without forcing an information bottleneck that discards fine-grained distributional structure.

5.3 Reproducibility

Full reproducibility is ensured through the following measures: the training seed is fixed to `42`, the train/validation split is stratified and performed before any fitting to prevent data leakage, evaluation is repeated over five seeds (`42, 12, 0, 7, 123`), DataLoader workers are seeded via a custom `seed_worker` function, all fitted preprocessing objects are saved to `artifacts/`, the best model weights are saved to `results/vae_model.pth`, and the training history and evaluation metrics are saved to `results/training_history.json` and `results/evaluation_results.csv` respectively.

6 Conclusion

We implemented a Variational Autoencoder with feature-specific decoder heads for synthetic tabular data generation on a mixed-type customer transaction dataset. The model achieves excellent categorical fidelity (all TVD < 0.05), moderately preserves correlation structure (MACD $= 0.0357$), and generates synthetic data with downstream utility better than the real-data baseline on the downstream classifier (TSTR accuracy $= 0.531$ vs. TRTR $= 0.498$). Numerical fidelity is mixed: `age` and `purchase_amount` pass the KS test, while `annual_income` still shows a significant shift.

Key architectural contributions include the use of LayerNorm for batch-size-independent normalisation, a KL warm-up schedule to prevent posterior collapse, a reduced $\beta_{\max} = 0.08$ to prioritise reconstruction, and a feature-weighted numerical loss that emphasises the harder-to-reconstruct `purchase_amount` column.

Future directions include exploring β-VAE annealing schedules, flow-based normalising priors in the latent space, or dedicated tabular architectures such as TVAE or CTGAN, which model continuous features with mode-specific Gaussian mixtures and can better capture irregular marginal distributions.

Output Files

Table 8: Saved Artefacts

File	Description
`results/synthetic_transactions_vae_final.csv`	1,500 synthetic transaction records
`results/vae_model.pth`	Best VAE model weights (PyTorch state dict)
`results/training_history.json`	Per-epoch loss components (train & val)
`results/evaluation_results.csv`	TSTR/TRTR metrics across 5 seeds
`SyntheticDataGen_final.ipynb`	Complete reproducible notebook